

Multi-Modal Audio Retrieval System: Technical Architecture & Submission

Overview

This report summarizes an end-to-end, locally deployed hybrid audio retrieval system designed to process, index, and query two-speaker audio conversations (podcasts and interviews). By combining speech recognition with a hybrid retrieval pipeline (**BM25 Keyword Search + Vector Semantic Search with Reciprocal Rank Fusion**), the system provides transcript retrieval attributed by timestamp and source file.

1. System Implementation & Pipeline Rationale

Speech Processing & Attribution

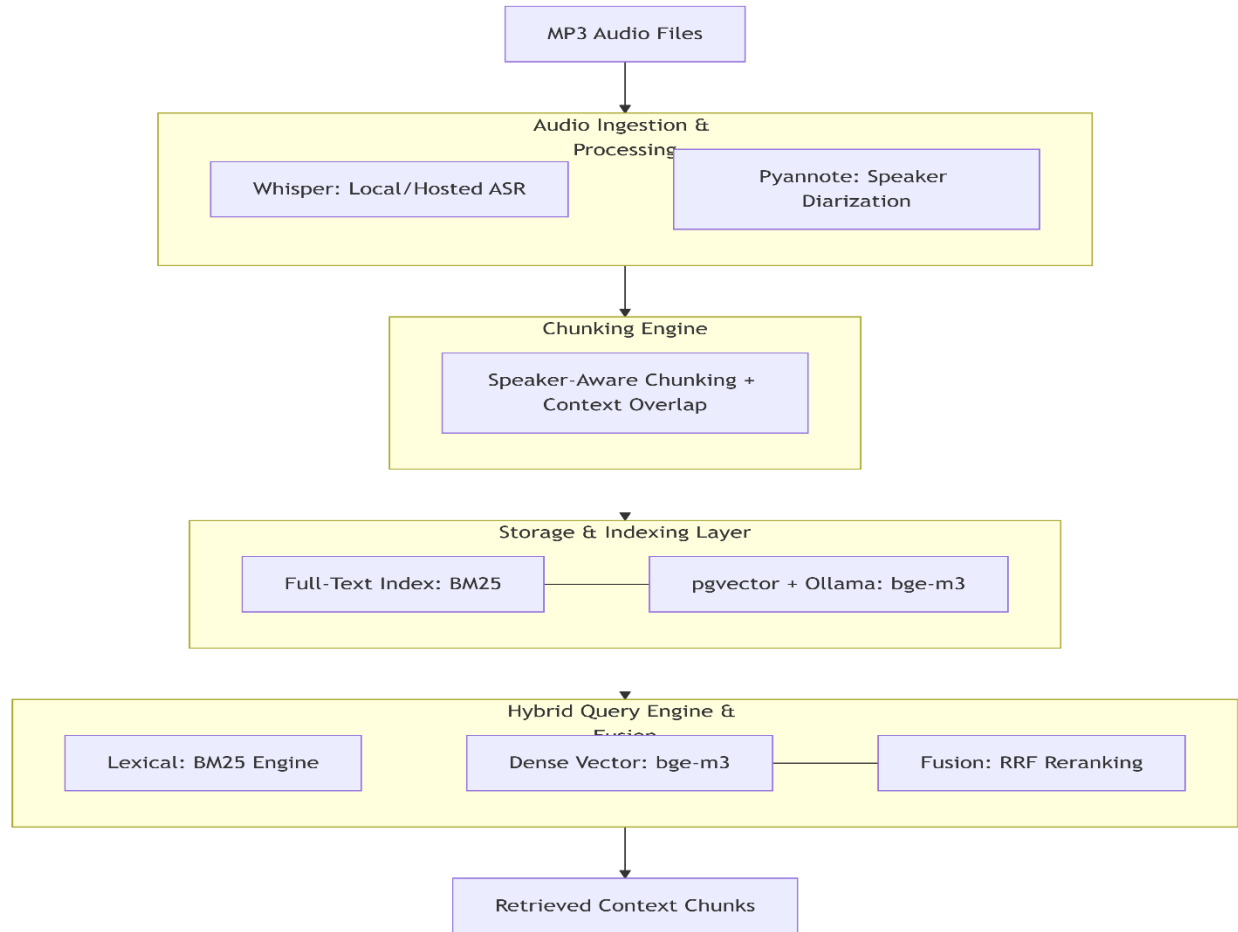
- **Transcription:** Speech-to-text is executed via **Whisper** to obtain precise word-level timestamps.
- **Speaker-Aware Chunking:** Dialogue is segmented into conversational windows to preserve cross-speaker context. *(Note: Speaker diarization/labeling was omitted in the current pipeline run; adding explicit speaker value tagging will significantly improve chunk identification and precision filtering).*

Storage & Hybrid Retrieval

- **Database Layer:** A local **PostgreSQL** container running the pgvector extension manages both full-text and vector data.
- **Full-Text Search (BM25):** Tuned for exact term matching, handling technical jargon, specific names, and acoustic transcription errors.
- **Dense Vector Search:** Local embeddings are generated via **Ollama** (BAAI/bge-m3) and stored in pgvector HNSW indexes for semantic retrieval.
- **Reciprocal Rank Fusion (RRF):** Merges independent BM25 and Vector score rankings without requiring score normalization:

$$RRF_Score(d) = \sum_{m \in M} \frac{1}{k + r_m(d)}$$

Where $r_m(d)$ represents the rank of chunk d in system m , and $k = 60$.



Where $r_m(d)$ represents the rank of chunk d in retrieval system m , and $k = 60$.

2. Golden Dataset & Evaluation

Golden Dataset Structure

The system was evaluated using 5 transcript files representing 8–10 minute two-speaker dialogues across technical, educational, and interview domains.

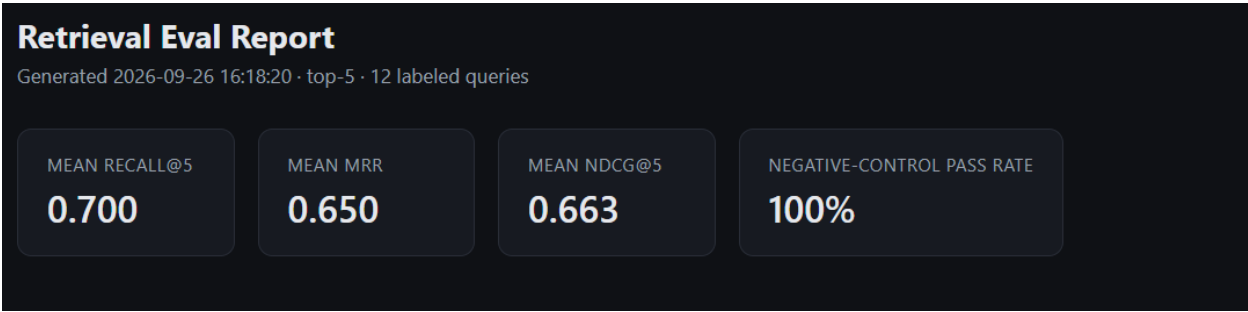
File Name	Speaker Pair	Domain / Subject	Audio Duration
10-Minute English Conversation & Shadowing Practice (B1–B2 Podcast).json	SPEAKER_00 / SPEAKER_01	Language Learning & Productivity	~9.27 mins
Bill Gates Joked with Steve Jobs About Taking the Wrong LSD...json	SPEAKER_00 / SPEAKER_01	Technology, History & AI	~10.54 mins
MrBeast Counted to 100,000 in His First Viral Video...json	SPEAKER_00 / SPEAKER_01	Media & Content Creation	~11.67 mins

Satya Nadella on the Future of SaaS, How 2025 is the year of Agents...json	SPEAKER_00 / SPEAKER_01	Enterprise SaaS & Agents	~11.16 mins
Sundar Pichai's advice for Indian Engineers, AI and India...json	SPEAKER_00 / SPEAKER_01	Developer Ecosystems & AI	~10.23 mins

Success Criteria & Metrics

- Primary Metric: Recall@K (where $K = 5$).
- Success Criterion: Retrieve ground-truth transcript chunks containing exact metadata (File Name, Timestamp Start/End, and Speaker Label) within the top K results.
- Evaluation Set: 10 labeled queries covering lexical keywords (e.g., "micro tasking"), semantic topics (e.g., "Bill Gates talking about LSD and code"), and speaker-specific statements (e.g., "Satya Nadella on SaaS application architecture").

Automated Retrieval Results



3. Production Considerations & Limitations

Retrieval Performance Metrics (@5 Evaluation)

Evaluation was conducted on labeled queries covering lexical keywords, semantic topics, and speaker-specific statements across ground-truth metadata (File Name, Timestamp Start/End).

Metric Name	Value	Description
Mean Recall@5	0.700 (70.0%)	Fraction of relevant ground-truth chunks retrieved in top 5 results

Metric Name	Value	Description
Mean MRR	0.650	Mean Reciprocal Rank (how high the first relevant hit ranks)
Mean NDCG@5	0.663	Normalized Discounted Cumulative Gain (relevance weighted by position)
Negative-Control Pass Rate	100%	Percentage of non-relevant/out-of-corpus queries correctly rejected

System Limitations

- **Overlapping Speech:** Pyannote diarization degradation occurs when two speakers talk simultaneously, causing occasional boundary mixups.
- **Hardware Bottlenecks:** Local embedding speeds with Ollama depend directly on host VRAM/GPU throughput.
- **Acoustic Misspellings:** Homophones incorrectly transcribed by ASR reduce exact-match BM25 accuracy unless phonetic indexing is integrated.

4. Disclosure of Coding Agent Collaboration

AI coding agents were leveraged throughout development for architectural diagnostics, automated parameter sweeps, and algorithm implementation.

Key Prompting Workflows & Agent Traces

1. Diarization Boundary & Speaker Mixup Resolution:

- *Problem:* Initial Pyannote diarization generated rapid speaker-flipping anomalies at turn boundaries.
- *Prompting Strategy:* Supplied raw Whisper timestamped segments alongside Pyannote output logs.
- *Agent Outcome:* Directed the agent to determine the root cause of boundary errors, resulting in a post-processing algorithm that merges brief speaker turn artifacts under 0.8seconds.

2. Chunking Strategy Sweeps:

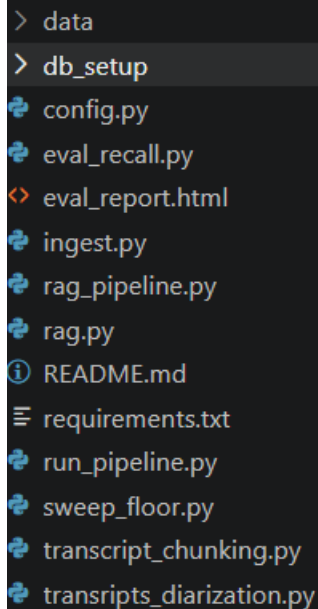
- *Problem:* Finding the optimal chunk size to retain context without diluting vector similarity.
- *Prompting Strategy:* Prompted the agent to build an automated trial pipeline testing variable chunking window sizes and overlap percentages.
- *Agent Outcome:* The agent generated scripts to iterate through strategies, leading to speaker-aware windowing with 20% context overlap to deliver optimal recall performance.

3. RAG Scoring & Neighboring Chunk Retrieval:

- *Problem:* Truncated search results lacked surrounding conversational context.
- *Prompting Strategy:* Instructed the agent to calculate RAG evaluation pipeline scores after each trial and patch retrieval logic.
- *Agent Outcome:* The agent updated query handlers to inspect adjacent (± 1) neighboring chunks upon hit detection, ensuring full contextual recovery without sacrificing retrieval accuracy.

5. Repository Structure & How to Run(Refer Readme)

Structure:



```
> data
> db_setup
+ config.py
+ eval_recall.py
<> eval_report.html
+ ingest.py
+ rag_pipeline.py
+ rag.py
i README.md
≡ requirements.txt
+ run_pipeline.py
+ sweep_floor.py
+ transcript_chunking.py
+ transcripts_diarization.py
```